Monitoramento de ransomware em nível de kernel utilizando eBPF e análise de entropia

Segunda apresentação: andamento desde agosto de 2026

Aluno	Kauã Junior Silva Soares, RA 231024061
Orientador	Prof. Dr. Kelton Augusto Pontara da Costa
Curso	Bacharelado em Ciência da Computação, UNESP Bauru

Contexto

Um caso recente de ransomware contra servidores, documentado por fonte oficial.

Em fevereiro de 2023, a CISA e o FBI publicaram o alerta AA23-039A sobre o ESXiArgs. Segundo o alerta, a campanha comprometeu mais de 3.800 servidores VMware ESXi e cifrou os arquivos de configuração das máquinas virtuais, o que pode deixar as máquinas inutilizáveis.

Esse tipo de ataque acontece no sistema de arquivos do servidor: um processo sobrescreve muitos arquivos em pouco tempo. É esse comportamento que o detector observa no kernel Linux.



Alerta AA23-039A, CISA e FBI, 8 de fevereiro de 2023. Captura de cisa.gov feita em 9 de novembro de 2026.

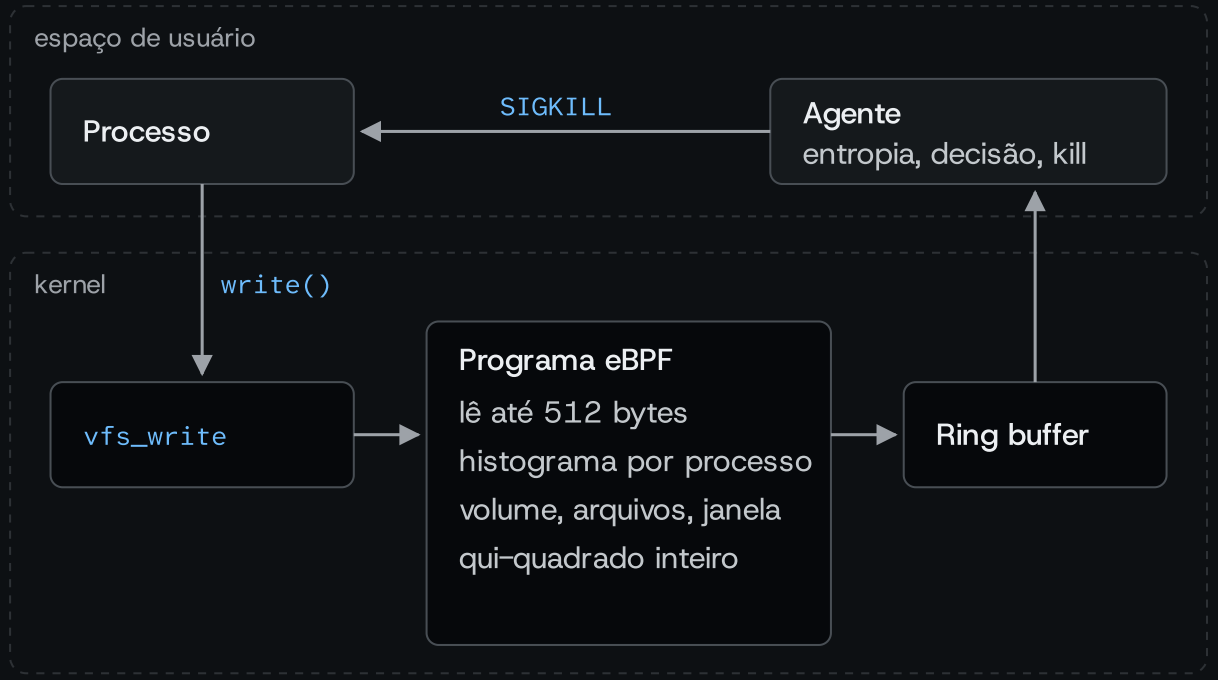
Funcionamento do detector

Revisão da arquitetura apresentada em agosto, já com as mudanças feitas desde então.

Toda escrita em arquivo no Linux passa pela função `vfs_write` do kernel. Um programa eBPF preso a ela por um kprobe lê o começo de cada escrita e conta, por processo, quantas vezes cada valor de byte apareceu.

Quando um processo passa de 64 KiB escritos em 10 arquivos diferentes dentro de 10 segundos, o histograma sobe para o agente em espaço de usuário, que calcula a entropia e decide se mata o processo.

Analogia. Funciona como um fiscal parado na única porta de saída de um depósito. Ele não precisa percorrer os corredores, porque tudo o que sai passa por ele.



Caminho de uma escrita até a decisão. O programa eBPF está em `src/bpf/ransomware_detect.bpf.c` e o agente em `src/userspace/main.cpp`.

Pendências da primeira apresentação

A apresentação de agosto terminou com três itens em aberto. Os slides seguintes seguem essa ordem.

Item	Situação em agosto	Situação agora	
Calibrar os limiares	O limiar de 7,8 vinha do ShieldFS e não tinha sido testado contra uso legítimo.	Calibrado em 40 execuções. Durante a preparação desta apresentação encontrei uma limitação com arquivos comprimidos grandes.	feito
Medir o custo	Sem medição.	Medido com fio em quatro cargas, em disco e em memória.	feito
Testar amostras reais	Planejado para VM isolada.	Ainda não começou. Depende da VM e do aval do orientador.	pendente

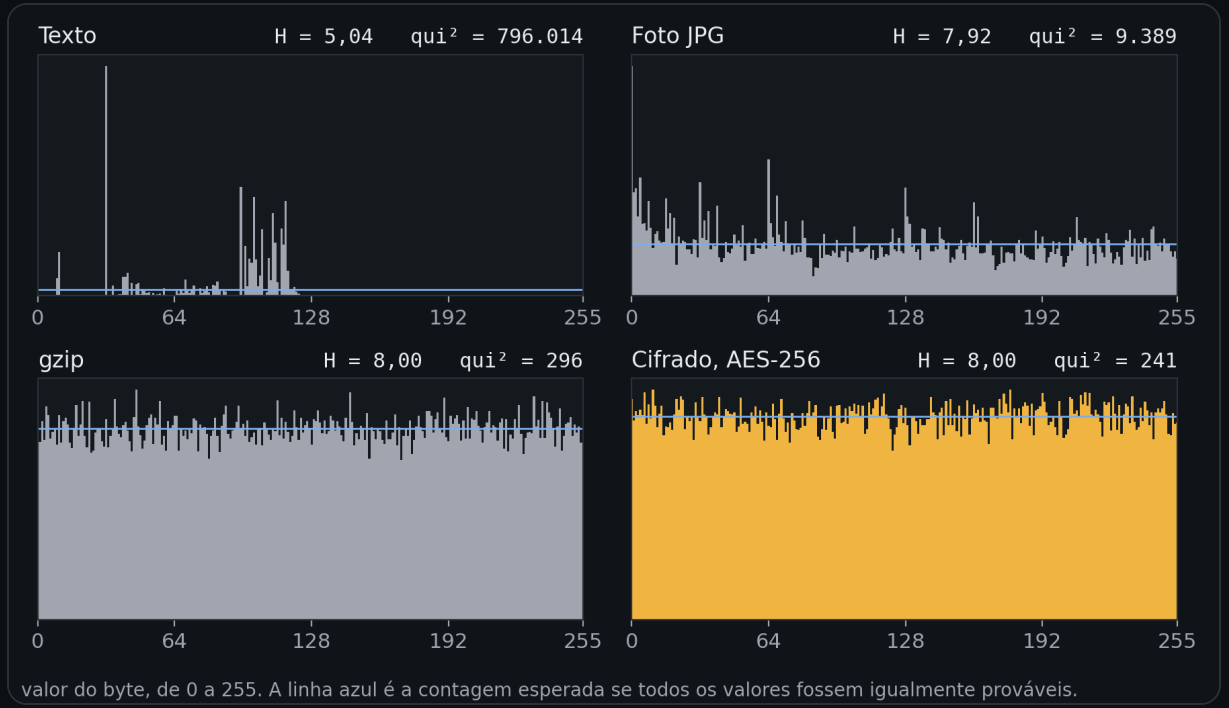
Além desses itens, a revisão do código mostrou três defeitos no programa eBPF e dois erros nas medições de agosto, que também estão nesta apresentação.

Entropia e qui-quadrado

As duas medidas que o detector aplica ao histograma de bytes, em quatro arquivos reais.

A entropia de Shannon mede o quanto cada byte é imprevisível, de 0 a 8 bits. O qui-quadrado mede o quanto a contagem de cada valor se afasta da contagem esperada se os 256 valores fossem igualmente prováveis. Para dado cifrado, o esperado fica perto de 255.

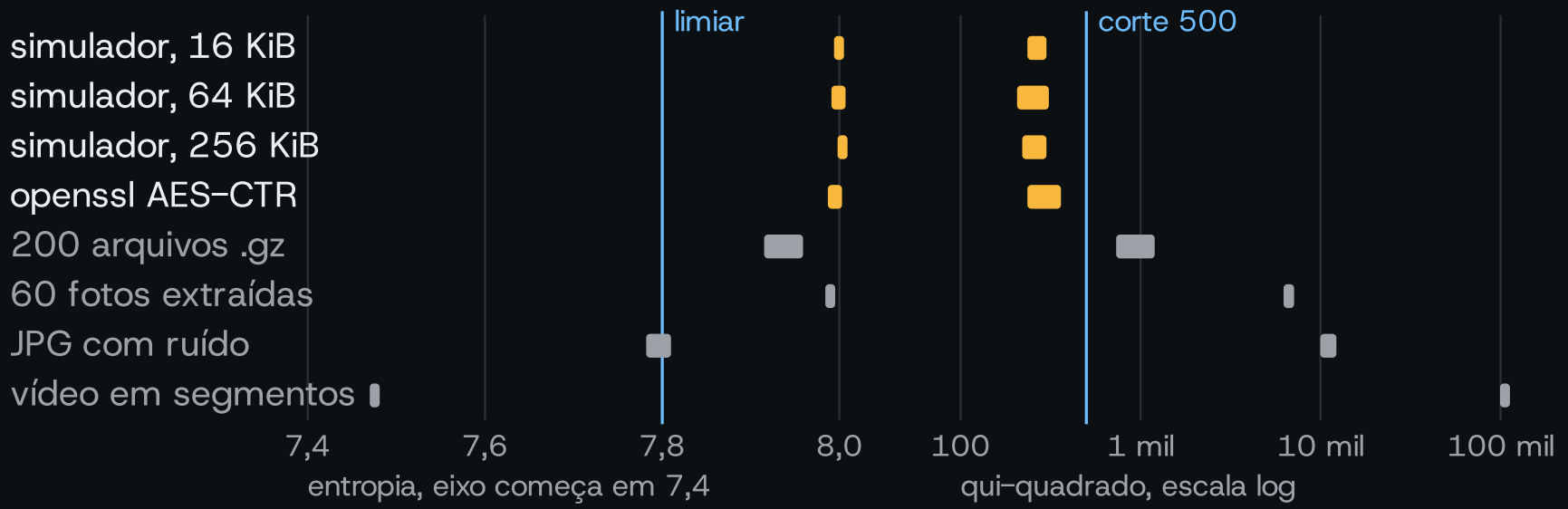
Exemplo. Para saber se um dado é honesto, joga-se o dado 600 vezes e conta-se cada face. Se todas as faces aparecem mas uma sai 150 vezes e outra 50, o dado é viciado. A entropia percebe que todas as faces apareceram. O qui-quadrado percebe a proporção errada.



64 KiB de cada arquivo. Texto de `/usr/include`, foto de `/usr/share/backgrounds` e gzip tirados do meio do arquivo, cifra AES-256 do mesmo texto com openssl. No trecho de gzip as duas medidas ficam perto das da cifra, o que volta no slide 7.

Calibração do critério de decisão

Detector em modo monitor durante 13 cargas, 3 vezes cada, mais 60 segundos de máquina ociosa. Nove cargas são uso legítimo, como compactar, extrair fotos e gerar vídeo. Quatro cifram arquivos.



Faixa de cada carga nas 3 execuções. Pela entropia, as fotos extraídas ficam a 0,012 bit da cifra. Pelo qui-quadrado, a cifra vai até 361 e nenhuma carga benigna da calibração fica abaixo de 731.

Entropia ≥ 7,8: alarme falso em 6 de 40 execuções, todas de gzip e fotos.

Entropia corrigida ≥ 7,8 e qui-quadrado ≤ 500: 0 de 40. As duas regras detectaram todas as cifragens.

Limitação: arquivos comprimidos grandes

Caso testado depois da calibração, ao preparar esta apresentação, e que a regra atual não resolve.

Nas cargas da calibração os arquivos comprimidos eram pequenos, e a amostra de 512 bytes pegava o cabeçalho e as tabelas do gzip. Em arquivos grandes, a amostra cai no meio do fluxo comprimido, que é quase uniforme.

Testei um processo comprimindo 12 arquivos de cerca de 2 MiB cada, emulando o detector sobre o strace. O gzip e o zstd geraram alarme falso. O xz escapou só porque levou mais de 10 segundos para chegar a 10 arquivos.

Uma saída que pretendo testar é diferenciar quem sobrescreve os arquivos que acabou de ler, como faz um ransomware, de quem cria arquivos novos, como faz um compressor.

Compressor	Entropia corrigida	Qui-quadrado	Resultado
gzip	7,995	335	alarme falso
zstd	7,998	332	alarme falso
xz	sem evento	sem evento	fora da janela

12 arquivos de texto de 2 MiB, um processo por compressor, regra atual com janela de 10 s e 10 arquivos.

Regra de decisão

Critérios usados em agosto e critérios atuais. O processo é morto quando todos são atendidos ao mesmo tempo.

Critério	Agosto	Agora
Entropia	~~≥ 7,8~~	corrigida, ≥ 7,8
Qui-quadrado	não havia	≤ 500
Janela de tempo	não havia	10 s
Volume escrito	≥ 64 KiB	≥ 64 KiB
Arquivos distintos	≥ 10	≥ 10

A correção de Miller-Madow soma à entropia um termo que depende de quantos valores diferentes apareceram e de quantos bytes foram amostrados. Com poucos bytes, alguns valores não aparecem e a entropia sai menor do que é: dado aleatório com 512 bytes dava 7,59 e não passava do limiar.

Analogia. Numa pesquisa eleitoral com 20 entrevistados, um candidato com 2% das intenções de voto pode não aparecer nenhuma vez. A correção estima o que ficou de fora pelo tamanho da amostra.

O corte de 500 no qui-quadrado fica entre o maior valor de cifra, 361, e o menor valor benigno, 731.

Correções no programa eBPF

Três defeitos encontrados na revisão do código que roda no kernel.

Defeito	Efeito	Correção
Pipe e socket contavam como arquivo	Conexões TLS e SSH transmitem dado cifrado o tempo todo e cumpriam os critérios.	Só arquivo regular entra na conta.
O mapa de processos nunca liberava espaço	Com 1024 posições e nenhuma apagada na saída do processo, o detector deixava de observar processos novos depois de 1024 processos.	Mapa do tipo LRU, com a entrada apagada quando o processo termina.
Não havia janela de tempo	Um processo comum podia juntar 10 arquivos ao longo de horas e ser tratado como cifragem em massa.	Os critérios valem dentro de 10 segundos.

Analogia para o mapa. Um estacionamento de 1024 vagas em que nenhum carro sai. Depois de lotado, ninguém mais entra.

```
if((modo & S_IFMT) != S_IFREG)
    return 0;
```

Filtro de arquivo regular, `src/bpf/ransomware_detect.bpf.c`, linha 144.

Erros de medição corrigidos

Dois problemas nos testes de agosto, que afetavam os números apresentados naquela ocasião.

Entropia do simulador abaixo do esperado

Os 8 alertas de agosto mostravam entre 7,92 e 7,93 bits por byte. O simulador gravava o próprio log num arquivo comum, e as linhas de texto desse log entravam no histograma do mesmo processo. Com o log desviado, o valor passou a 7,9996.

Carga do benchmark invisível ao detector

O fio estava configurado com libaio, que envia as escritas por `io_submit`. Esse caminho não passa por `vfs_write`, e o detector não via a carga que estava sendo medida. Troquei para psync.

> **Analogia.** Era como medir o movimento de uma loja com a câmera da porta da frente enquanto a mercadoria entrava pela garagem.

Os 8 alertas antigos estão em bench/results/legado

Tempo até o bloqueio

Simulador cifrando 300 arquivos, com o detector em modo de bloqueio, 5 execuções por tamanho de arquivo.

Em todas as 15 execuções o processo foi morto no décimo arquivo, que é o mínimo do critério. A perda é definida por esse número e não pela velocidade do detector: entre o kernel e o agente o evento leva cerca de 50 µs.

Analogia. Um alarme regulado para tocar quando há fumaça em 10 cômodos toca sempre no décimo. Baixar para 5 faz tocar antes, mas também com a fumaça da cozinha.

```
$ xxd -l 16 benigno_0000.txt   # antes
00000000: 4c6f 7265 6d20 6970 7375 6d20 646f 6c6f  Lorem ipsum dolo
$ xxd -l 16 benigno_0000.txt   # depois do simulador
00000000: 38ed 3400 5cdd 6ca2 e4bc 21a6 6c21 9a31  8.4.\.l...!.l!.1

$ sudo ./ransomware_detect --kill
[ALERTA] PID=76257 PROC=sim_encrypt H=7.9996 bits/byte
         QUI2=258.1 BYTES=655360 ARQUIVOS=10
[BLOQUEIO] SIGKILL enviado para PID 76257
```

Saídas reais. O hexdump é de um arquivo antes e depois do `sim_encrypt`, e o alerta é do log `bench/results/kill/rapido_64k_0ms_1.log`, com a linha quebrada em duas para caber.

Tamanho de cada arquivo	Até o SIGKILL
16 KiB	13,4 ms
64 KiB	17,6 ms
256 KiB	31,6 ms

bench/results/latencia_kill.csv

Janela de tempo e ransomware lento

O que acontece quando o simulador espera entre um arquivo e outro.

A janela de 10 segundos limita o tempo que o detector espera para juntar os 10 arquivos. Um ransomware que cifra mais devagar que um arquivo por segundo não chega a cumprir o critério.

> **Analogia.** O radar de velocidade média mede o tempo entre dois pontos da estrada. Quem passa devagar o bastante não é multado, mesmo que tenha corrido em algum trecho.

Aumentar a janela pega o ransomware lento, mas também junta mais escrita legítima. Essa troca vai para a monografia como parâmetro a ser discutido.

Intervalo entre arquivos	Bloqueado	Arquivos perdidos
Nenhum	15 de 15	10 de 300
500 ms	3 de 3, aos 4,6 s	10 de 40
1500 ms	0 de 3	40 de 40

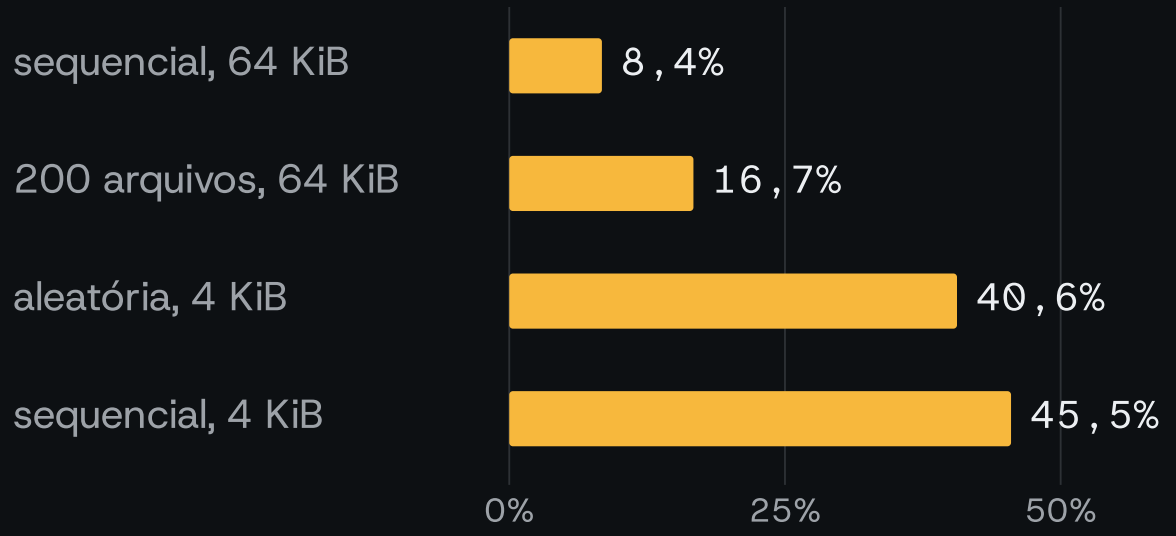
Custo de desempenho

Perda de vazão de escrita com o detector ativo, medida com fio em quatro cargas, 5 repetições cada.

A primeira medição, em disco, variou de 128 a 352 MB/s dentro do mesmo modo, e o custo do detector não aparecia no meio desse ruído. Repeti em `/dev/shm`, que fica na memória, onde sobra só o custo de CPU. É o pior caso.

Cada chamada do kprobe custa entre 1,03 e 1,40 µs, quase o mesmo valor em todas as cargas.

Analogia. É um pedágio de valor fixo. Numa escrita de 64 KiB, que leva cerca de 16 µs, quase não pesa. Numa de 4 KiB, que leva cerca de 2 µs, pesa bastante.



Vazão perdida em memória. Em disco a diferença ficou dentro da variação entre repetições.

Filtro de qui-quadrado no kernel

Mudança feita depois da medição de custo, para tirar do agente o trabalho com eventos que nunca seriam alarme.

Na carga de 200 arquivos, o fio mandava ao agente cerca de 18 mil eventos por segundo, nenhum com cara de cifra, e o agente gastava 41% de um núcleo só para descartá-los. Passei o cálculo do qui-quadrado para o kernel. Como o eBPF não tem ponto flutuante, a comparação foi reescrita só com inteiros.

Com o filtro, o kernel descartou entre 320 mil e 390 mil eventos por execução e não enviou nenhum. A CPU do agente caiu de 8,2 s para 0,016 s em cada execução de 20 s.

Analogia. É a triagem do pronto-socorro: só passa para o médico quem tem sintoma.

```
// qui2 <= max vira 256 * soma dos quadrados <= n * (n + max)
for(int i = 0; i < 256; i++) {
    __u64 c = st->histograma[i];
    n += c;
    soma_quad += c * c;
}
return 256 * soma_quad <= n * (n + cfg_max_qui2);

$ sudo ./ransomware_detect   # carga de 200 arquivos
[*] encerrando detector, eventos enviados=0
    descartados no kernel=389394
```

Trecho de `parece_uniforme`, linhas 76 a 94 do programa eBPF, e o fim de um log real da medição. Em 3.000 histogramas de teste, a conta inteira deu a mesma decisão da conta em ponto flutuante.

Próximas etapas

O que falta até a postagem da monografia, em 8 de janeiro de 2027, e a banca, de 18 a 21 de janeiro.

Etapa	Objetivo
Falso positivo de compressão	Separar o processo que sobrescreve os arquivos que leu do processo que cria arquivos novos, e repetir a calibração com arquivos comprimidos grandes.
Variantes de famílias reais	Simular comportamentos descritos em relatórios públicos: cifragem intermitente, cifragem só do início do arquivo, nota de resgate entre os arquivos, escrita por mmap e várias threads.
Amostras reais	Executar em VM isolada, sem rede e com snapshot, depois do aval do orientador.
Monografia	Resultados, limitações e a discussão sobre janela de tempo e número mínimo de arquivos.

Referências e perguntas

Código, scripts dos experimentos e resultados: github.com/Kauajr13/ebpf-ransomware-detector

CISA; FBI. *ESXiArgs Ransomware Virtual Machine Recovery Guidance*. Alerta AA23-039A, 8 fev. 2023. Disponível em: cisa.gov/news-events/cybersecurity-advisories/aa23-039a.

CONTINELLA, A. et al. ShieldFS: a self-healing, ransomware-aware filesystem. In: Annual Computer Security Applications Conference, 32., 2016. p. 336-347.

SCAIFE, N. et al. CryptoLock (and drop it): stopping ransomware attacks on user data. In: IEEE International Conference on Distributed Computing Systems, 36., 2016. p. 303-312.

MILLER, G. A. Note on the bias of information estimates. In: QUASTLER, H. (ed.). *Information theory in psychology: problems and methods*. Glencoe: Free Press, 1955. p. 95-100.

GREGG, B. *BPF performance tools: Linux system and application observability*. Boston: Addison-Wesley, 2019.

NAKRYIKO, A. BPF CO-RE (compile once, run everywhere). 2020. Disponível em: nakryiko.com/posts/bpf-core-reference-guide.

Foto usada no histograma do slide 5: *Clouds*, de Tibor Mokanszki, papel de parede do Ubuntu.